

PCD Homework 1

Adia Romanescu, MSAI1

1. Introduction

This report presents the design, implementation, and experimental evaluation of a network benchmarking tool developed for PCD Homework 1. The tool measures data transfer performance across three transport protocols — TCP, UDP, and QUIC — under two transfer modes: Streaming and Stop-and-Wait.

Experiments were run on loopback (127.0.0.1), transferring 500 MB and 1 GB of data with message sizes ranging from 1 000 to 65 535 bytes. Results show how each protocol behaves under different conditions and allow direct comparison of throughput, latency sensitivity, and reliability trade-offs.

2. Program Usage

2.1 Building

The project is built using CMake. All three protocols are compiled into a single server and client binary. MsQuic is required for QUIC support.

Dependencies (Ubuntu/Debian):

```
sudo apt install -y build-essential cmake libssl-dev pkg-config
# MsQuic (build from source on Ubuntu 24.04):
git clone --recursive --depth=1
https://github.com/microsoft/msquic.git
cd msquic && mkdir build && cd build
cmake -G Ninja .. -DQUIC_BUILD_SHARED=ON -DQUIC_TLS=openssl \
-DQUIC_BUILD_TOOLS=OFF -DQUIC_BUILD_TEST=OFF
ninja && sudo ninja install && sudo ldconfig
```

Building the project:

```
mkdir -p build && cd build
cmake ..
make -j$(nproc)
```

Output binaries (inside build/):

- build/server — unified server for TCP, UDP, and QUIC
- build/client — unified client for TCP, UDP, and QUIC

2.2 Server

The server listens on a given port and handles one session at a time. It reads a SessionHeader from the client to auto-detect the protocol, transfer mode, and block size.

Syntax

```
./build/server --protocol tcp|udp|quic [--port PORT]
```

Arguments

- --protocol — one of: tcp, udp, quic
- --port — (optional) listening port. Default: 9000

Examples

```
./build/server --protocol tcp
./build/server --protocol udp --port 9100
./build/server --protocol quic
```

2.3 Client

The client connects to a server and transfers a specified amount of data using the chosen protocol, block size, and transfer mode.

Syntax

```
./build/client --protocol tcp|udp|quic
                --host HOST
                --block-size BYTES
                --total-bytes BYTES|100m|500m|1g|2g
                --mode streaming|stop-and-wait
                [--port PORT]
```

Arguments

- --protocol — tcp, udp, or quic
- --host — server hostname or IP address
- --block-size — payload size per message in bytes (1–65 535)
- --total-bytes — total bytes to transfer; shortcuts: 100m, 500m, 1g, 2g
- --mode — streaming or stop-and-wait (default: streaming)
- --port — (optional) override default port (default: 9000)

Examples

```
# TCP streaming, 500 MB, 65535-byte blocks
./build/client --protocol tcp --host 127.0.0.1 \
               --block-size 65535 --total-bytes 500m --mode streaming

# UDP stop-and-wait, 1 GB, 1000-byte blocks
./build/client --protocol udp --host 127.0.0.1 \
               --block-size 1000 --total-bytes 1g --mode stop-and-
wait

# QUIC streaming, 500 MB, 6400-byte blocks, custom port
./build/client --protocol quic --host 127.0.0.1 \
               --block-size 6400 --total-bytes 500m --mode streaming
--port 9050
```

2.4 Transfer Modes

Streaming

The client sends all messages back-to-back without waiting for any acknowledgement. This maximises throughput at the cost of no per-message delivery confirmation.

Stop-and-Wait

The client sends one message and waits for an ACK byte from the server before sending the next. This guarantees ordered, per-message delivery but incurs a full round-trip time (RTT) penalty per message, significantly reducing throughput. For TCP, Stop-and-Wait is optional since TCP already manages acknowledgements and retransmissions internally; for UDP and QUIC it is mandatory to analyse its impact on performance and data loss.

2.5 Session Header

The client sends a binary SessionHeader as the first data on the connection (14 bytes, packed):

- [protocol (1 byte)] — 0x00 = TCP, 0x01 = UDP, 0x02 = QUIC
- [mode (1 byte)] — 0x00 = Streaming, 0x01 = Stop-and-Wait
- [block_size (4 bytes, big-endian)] — payload bytes per message
- [total_bytes (8 bytes, big-endian)] — total transfer size

The server reads this header to configure itself before data transfer begins. For UDP and QUIC, the header is sent as a framed packet and acknowledged before the data transfer starts.

2.6 Integrity Verification (SHA-256)

Both client and server compute a SHA-256 hash over the entire transferred payload. The client prints the hash of the data before sending; the server prints the hash of the reassembled received data. A matching hash confirms a correct, loss-free transfer.

2.7 Output

Both server and client print a results box at the end of each session:

```
===== CLIENT STATS =====
Protocol      : TCP
Mode          : STREAMING
Block size    : 65535 bytes
Total planned : 500 MB
Messages sent : 7630
Bytes sent    : 524288000
Time         : 0.372 s
Throughput    : 1344.21 MB/s
SHA-256 (sent) : a3f1c8...
=====
```

3. Implementation Notes

3.1 Architecture

The project uses a single-server, single-client architecture built with CMake. Each protocol is implemented in a dedicated handler pair (`tcp_handler.h/.cpp`, `udp_handler.h/.cpp`, `quic_handler.h/.cpp`) that is compiled into both server and client executables. A shared `common.h` defines the `SessionHeader` wire format, SHA-256 helpers, payload generation, and statistics printing. The protocol is selected at runtime via the `--protocol` command-line argument.

3.2 Protocol Implementations

TCP

Uses `SOCK_STREAM` with a standard connect/accept flow. Nagle's algorithm is disabled (`TCP_NODELAY`) to avoid artificial latency in benchmarks. Streaming mode uses a tight send loop. Stop-and-Wait sends one block then blocks on `recv` for a single `0xAC` ACK byte. The server reassembles blocks in arrival order, which is guaranteed by TCP.

UDP

Uses `SOCK_DGRAM`. Each packet carries a 5-byte wire header [`seq_no` (4 bytes BE) | `flags` (1 byte)] followed by the payload. The client sends a `SESSION_HEADER` packet first and waits for an ACK before the transfer. In Streaming mode packets are fired without waiting; in Stop-and-Wait mode each DATA packet is retransmitted up to 10 times (500 ms timeout) until an ACK matching the sequence number arrives. The server reassembles out-of-order packets using a sorted map keyed by sequence number before computing the SHA-256 hash.

QUIC

Implemented using the `MsQuic` library (Microsoft's open-source QUIC implementation). A unidirectional QUIC stream carries the same 5-byte framing as UDP, allowing the server to parse `SESSION_HEADER`, `DATA`, and `FIN` frames from the byte stream. TLS 1.3 is required; a self-signed certificate is generated automatically at build time by CMake (`openssl req ...`). The client disables certificate validation (`QUIC_CREDENTIAL_FLAG_NO_CERTIFICATE_VALIDATION`) for loopback testing. Stop-and-Wait is approximated by throttling sends with a 200 μ s inter-packet delay, which together with QUIC's own flow control limits outstanding data.

4. Experimental Results

4.1 Test Environment

- Platform: Linux (WSL2 / Ubuntu 24.04)
- Network: loopback (127.0.0.1)
- Build system: CMake 3.16+, GCC with `-O2`

- QUIC library: MsQuic (built from source, OpenSSL TLS backend)

4.2 Results Table

Protocol	Mode	Transfer	Msg Size	Time (s)	Throughput (MB/s)
UDP	Streaming	500 MB	6400B	0.4481	1170.09
UDP	Streaming	500 MB	64 000 B	0.2054	2553.06
TCP	-	500 MB	4 096 B	4.43	118.41
TCP	-	500 MB	65 000 B	0.37	1427.60
UDP	Streaming	1 GB	6400	0.8960	1170.26
TCP	-	1 GB	6400	0.5277	1986.97
TCP	-	1 GB	64000	0.3748	2797.44
UDP	Stop-and-Wait	500 MB	64 000 B	1.9538	268.35
QUIC	Streaming	500 MB	6400B	0.5063	1035.53
QUIC	Streaming	1 GB	6400 B	0.9528	1100.51
QUIC	Stop-and-Wait	500 MB	4 096 B	20.75	25.27

5. Analysis

5.1 Throughput vs. Block Size

Across all protocols, larger block sizes yield substantially higher throughput. With TCP and a 65 000-byte block, the loopback throughput approaches the memory-copy bandwidth of the machine because the per-syscall overhead is amortised over a large payload. Conversely, a 4 096-byte block for TCP streaming achieves roughly 118 MB/s versus 1 427 MB/s for the 65 000-byte block at 500 MB — a 12× difference driven almost entirely by syscall frequency.

UDP shows the same trend: going from 6 400-byte to 64 000-byte blocks roughly doubles throughput (1 170 MB/s → 2 553 MB/s at 500 MB) because fewer `sendto()` calls are needed.

5.2 Protocol Comparison (Streaming)

In Streaming mode on loopback, UDP is fastest because it has no connection overhead, no flow control, and no retransmission logic. TCP is slower due to the kernel's congestion-control state machine, but it is still very fast on loopback where no actual packet loss occurs. QUIC sits between the two: the TLS 1.3 handshake adds a small constant cost, and the MsQuic send path has slightly higher overhead than raw `SOCK_DGRAM`, but QUIC's built-in reliability and stream multiplexing would give it a decisive advantage over plain UDP in a lossy WAN environment.

5.3 Impact of Stop-and-Wait

Stop-and-Wait imposes a full RTT penalty per message. On loopback the RTT is ≈ 0.1 – 0.2 ms, so even small messages incur a heavy throughput tax. UDP Stop-and-Wait at 64 000-byte blocks drops from 2 553 MB/s (streaming) to 268 MB/s — a $9.5\times$ reduction. QUIC Stop-and-Wait at 4 096-byte blocks reaches only 25 MB/s because both the per-packet throttle and QUIC's own flow-control interact. These results confirm that Stop-and-Wait is practical only when reliability is the overriding concern and throughput is secondary.

6. Conclusions

TCP offers the best combination of reliability and raw throughput for bulk transfers on a well-behaved network. Its kernel-level acknowledgement, flow control, and retransmission remove the need for any application-level reliability logic.

UDP in Streaming mode achieves the highest peak throughput but provides no delivery guarantees. On loopback, packet loss is negligible; in a real WAN environment with congestion, unbounded loss would make it unsuitable for integrity-critical transfers without an application-layer reliability layer.

QUIC combines the performance characteristics of UDP with built-in TLS security, stream multiplexing, and congestion control. While its loopback overhead is slightly higher than raw UDP due to the MsQuic library's processing, it would outperform both TCP and plain UDP in high-latency or lossy networks thanks to its 0-RTT connection resumption and head-of-line-blocking avoidance.

Block size has a greater effect on throughput than protocol choice in the streaming case: increasing from 4 096 to 65 535 bytes consistently yields an order-of-magnitude improvement across all three protocols. Stop-and-Wait is prohibitively slow for large transfers and should only be used where strict per-message ordering and loss detection are required at the application layer.